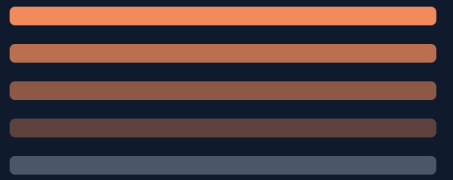
REFERENCE ARCHITECTURE · DRAFT FOR REVIEW



AI-ready applications

An AI foundation for the software a business already runs: open models, private data, agent-ready, sized to actual use.

[Company name]

October 2026

THE PROBLEM

The pilot-to-production gap

78%

of enterprises have an AI agent pilot running

14%

have scaled one across the organisation

What leaders say stops them



Model capability is not on the list. These are architecture and operating problems.

Source: Digital Applied survey of 650 enterprise technology leaders, February to March 2026.

THE OFFER

We add an AI foundation beside the application a client already runs, inside their own environment, without rewriting it.

One foundation

Search, memory, model serving and agent access, built once and reused by every later AI feature.

The client’s control

Open-weight models on the client's hardware or private cloud. Data does not leave unless they choose.

Sized to use

Starts on a small footprint with one use case. Capacity is added when measured usage calls for it.

ARCHITECTURE

Five design principles

Add, without rewriting	The foundation connects through the application's existing APIs, database and login. The application's own code stays as it is.
Any model, one interface	Every model call goes through one gateway, so a model can be swapped without touching the application.
Data stays inside	Models, search index and logs run in the client's data centre or private cloud. Nothing leaves unless the client chooses.
Scale when usage asks	Start on one GPU node. Add capacity when measured usage calls for it, and release it when demand drops.
Measured from day one	Evaluation tests, request tracing and cost tracking are in place before the first feature ships.

ARCHITECTURE

The reference architecture

Channels	Existing app screens, an embedded assistant, and outside AI agents
Agent interface	MCP server over the app's APIs, WebMCP tools in the browser, every call under the user's own login
Orchestration	Agent runtime, multi-step workflows, human approval before any write action
AI gateway	One endpoint for every model: routing, caching, token budgets, guardrails
Knowledge and memory	Ingestion pipeline, hybrid search with reranking, session and long-term memory
Model serving	Open-weight language, embedding and reranker models on the client's GPUs, with an optional private endpoint
Existing application	Database, APIs, documents and login stay as they are

Security and identity

Single sign-on and role-based access

Personal data masking

Injection checks

Audit log

Observability and evals

Trace for every request

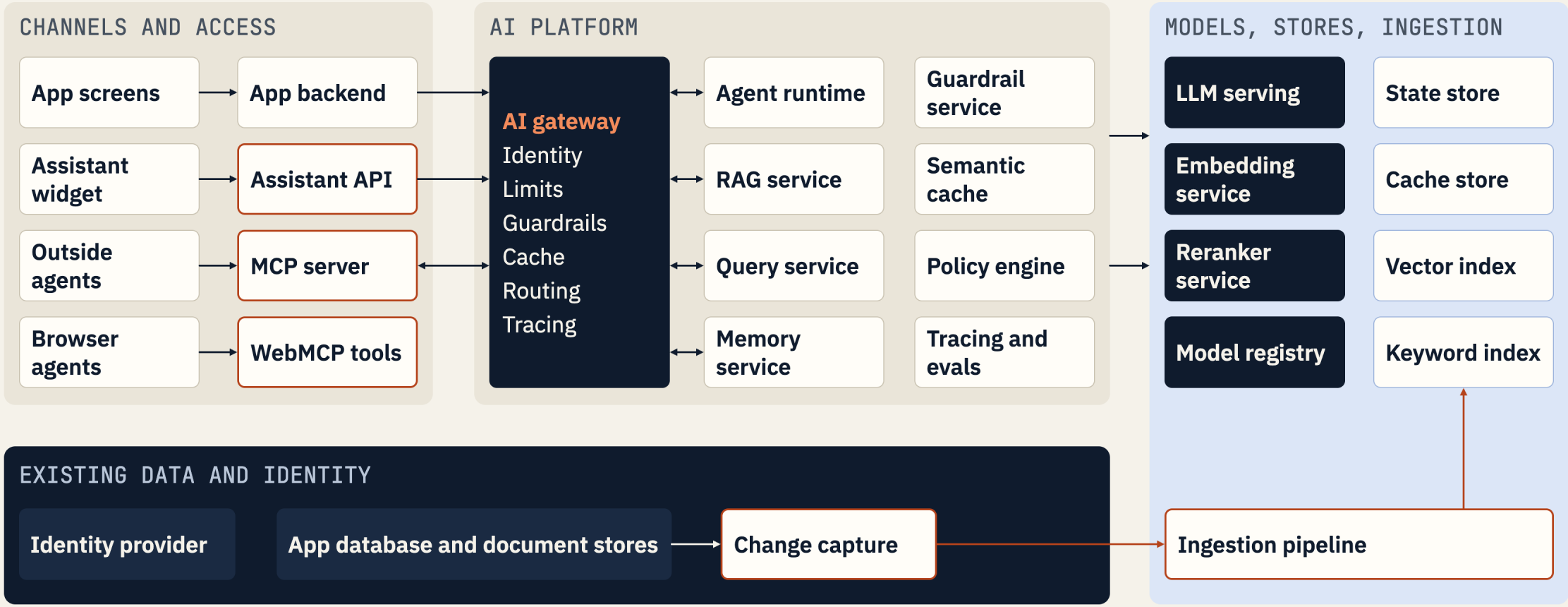
Cost and latency per feature

Test suite on every release

Quality scores over time

ARCHITECTURE

System architecture



ARCHITECTURE

The path of one request

01

Ask

A user in the app, or an outside agent through MCP, sends a request.

02

Check

Identity is confirmed. The input is screened for injected instructions and personal data.

03

Reuse

The cache answers repeated or near-identical questions without calling a model.

04

Retrieve

Hybrid search pulls only the records this user is allowed to see.

05

Route

The gateway sends the request to the smallest model that passes the evaluation for that task.

06

Act and record

Tools run through MCP, writes wait for approval, and the full trace is logged and scored.

ARCHITECTURE

Model strategy

Model tier	Typical work	Where it runs
Small open-weight model	Routing, classification, extraction	One shared GPU
Mid-size open-weight model	Most chat, search answers and tool calls	Client GPUs
Large model, or a private enterprise endpoint	Long multi-step agent tasks and hard reasoning	GPU cluster, or the client's private cloud tenancy
Embedding and reranker models	Semantic search	Same GPUs, small footprint

Open-weight first, not open-weight only. Open models still trail the leading closed models on long, multi-step agent work. The gateway lets each workload use whichever model passes its evaluation, and every model licence is checked before deployment.

Families reviewed at assessment include Qwen, Llama, Mistral, Gemma and DeepSeek. The choice is made per client.

ARCHITECTURE

Agent interface: MCP and WebMCP

MCP server

In production use today

Wraps the application's existing APIs as typed tools on the server side.

Any MCP-capable assistant or agent can discover and call them.

Each call carries the signed-in user's permissions.

WebMCP

Emerging, experimental

The web front end registers tools that a browser agent calls directly, with no screen scraping.

Available as a Chrome origin trial. The specification is a W3C community group draft, not yet a standard.

We ship it as an add-on: the page behaves the same where it is absent.

Both expose the actions the application already offers, under the permissions it already enforces.

WebMCP status as of October 2026. Sources: Chrome for Developers; InfoQ, June 2026.

ARCHITECTURE

Cost controls in the gateway

Prefix caching

System prompts and tool definitions that repeat are cached by the model server, not recomputed.

Semantic cache

Questions that mean the same thing return a stored answer, checked against the user's permissions.

Model routing

Simple requests go to a small model. Only the hard ones reach the large model.

Context trimming

Reranked retrieval and summarised history keep each prompt to what the task needs.

Token budgets

Limits and usage reports for each team, feature and user.

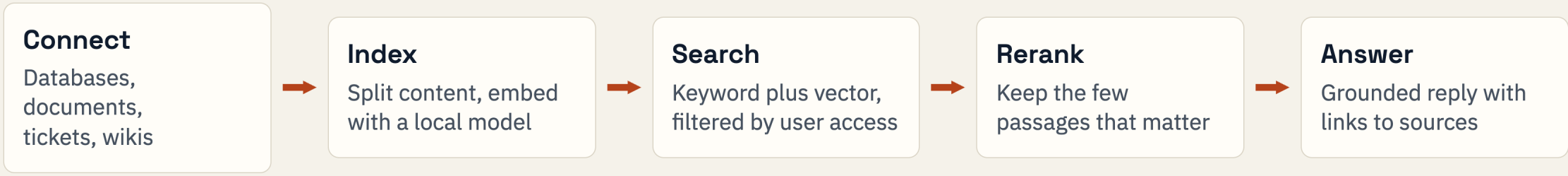
Batch and async

Work that nobody is waiting on runs off-peak on spare capacity.

ARCHITECTURE

Knowledge and memory

SEARCH PIPELINE



MEMORY

Session

The current conversation, summarised as it grows.

Long-term

Facts and preferences for each user, stored with consent and an expiry date.

Shared knowledge

The search index above, refreshed as the source systems change.

ARCHITECTURE

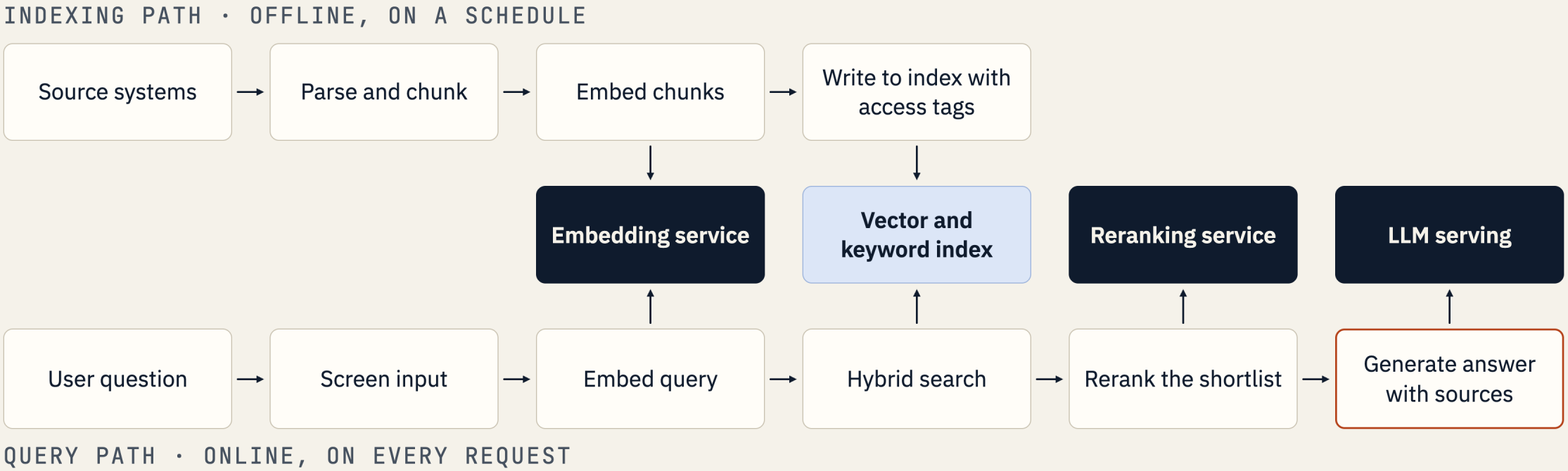
Controls for security review

Control	How it works
Identity	Agents act as the signed-in user. There are no shared service accounts.
Guardrails	Inputs are screened for injected instructions, personal data is masked, and outputs are validated against a schema.
Approval gates	Reads run freely. Writes, payments and deletions wait for a person.
Evaluation	A test set built from real client questions runs before every release.
Audit trail	Every prompt, retrieved record, tool call and answer is logged and traceable.

These are built before the first agent, so the security team reviews a finished design.

DEEP DIVE

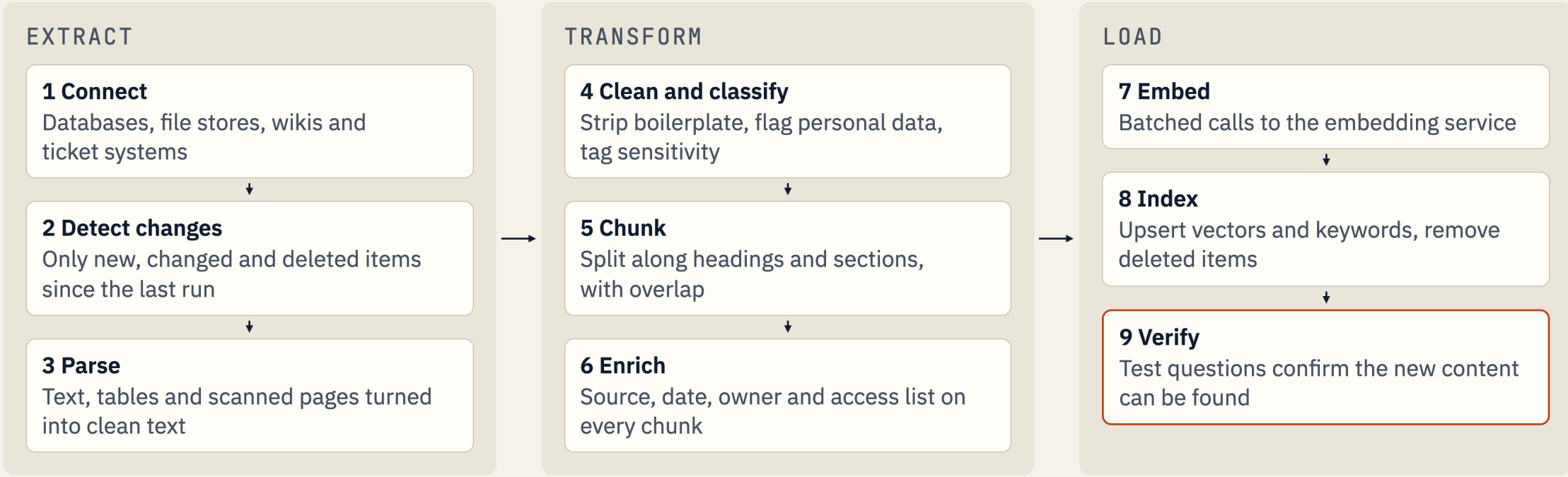
RAG architecture



Dark boxes are model services. The blue box is the one store both paths share: indexing writes to it, queries read from it.

DEEP DIVE

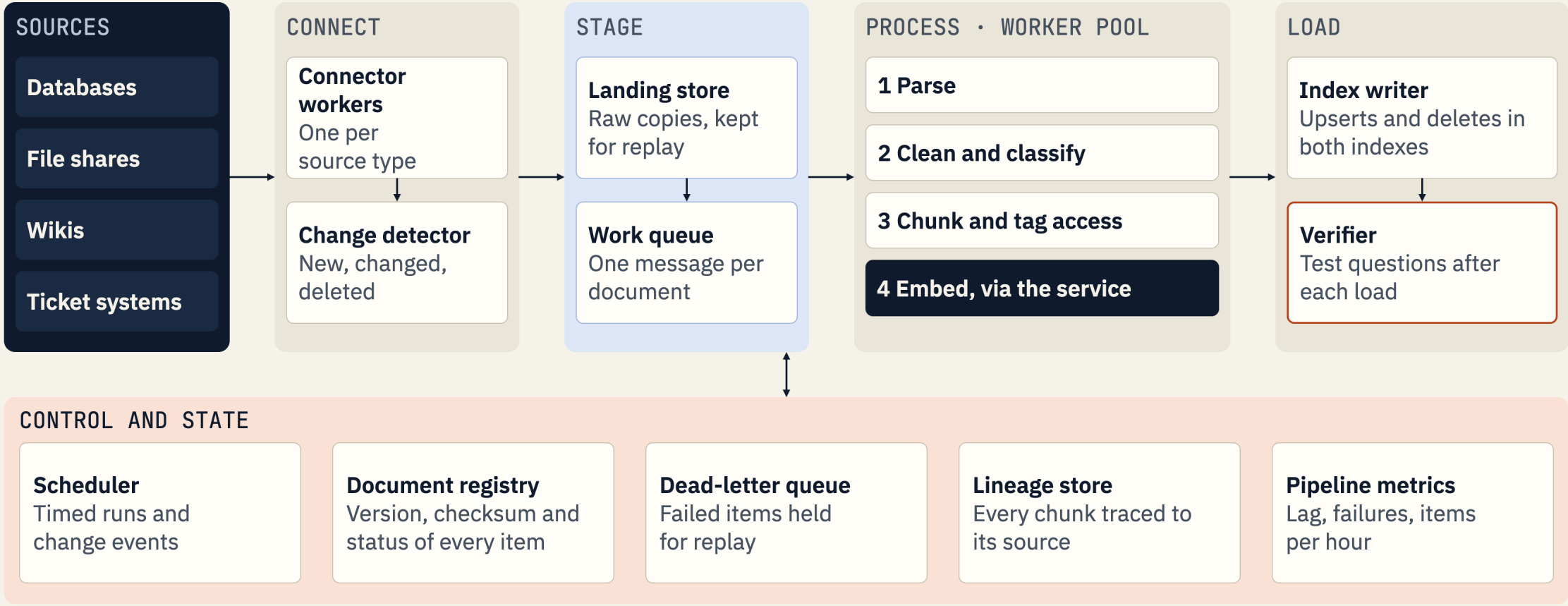
RAG ingestion pipeline



Across every stage: scheduled and event-driven runs, retries with a dead-letter queue, and lineage from every chunk back to its source record.

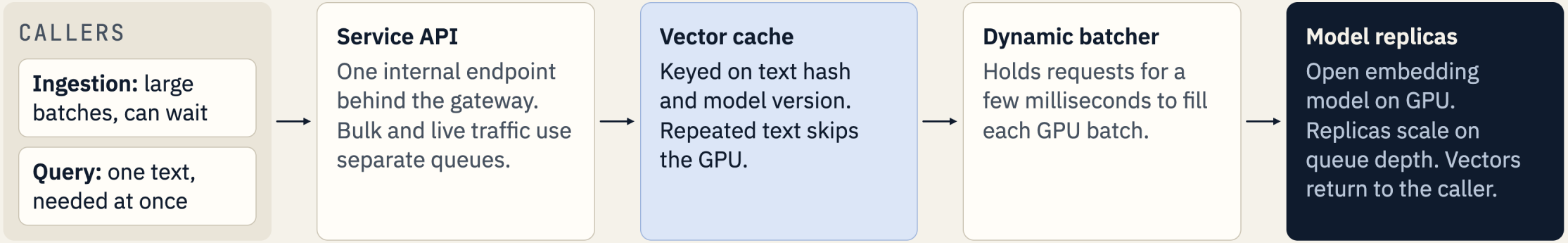
ARCHITECTURE

Ingestion platform architecture



DEEP DIVE

Embedding model service



One model version per index

Vectors from different models cannot be compared. Changing the model means a full re-embed.

Same model on both paths

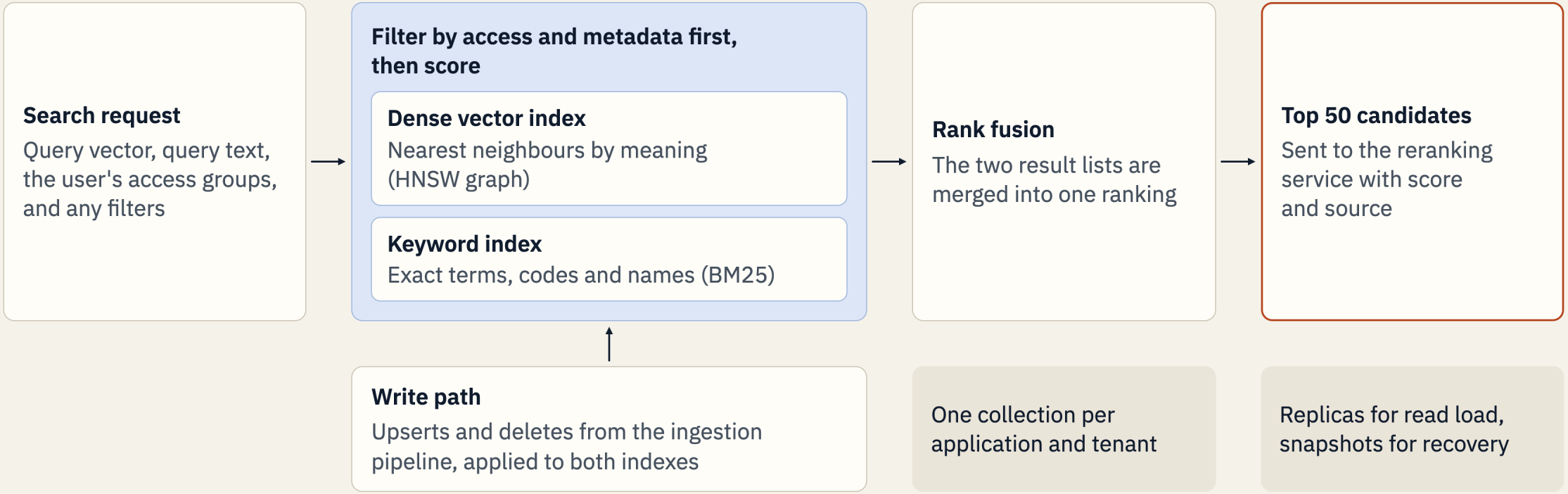
Documents and questions are embedded by the same model with the same settings.

Small footprint

Embedding models are usually much smaller than the chat model and can share a GPU with the reranker.

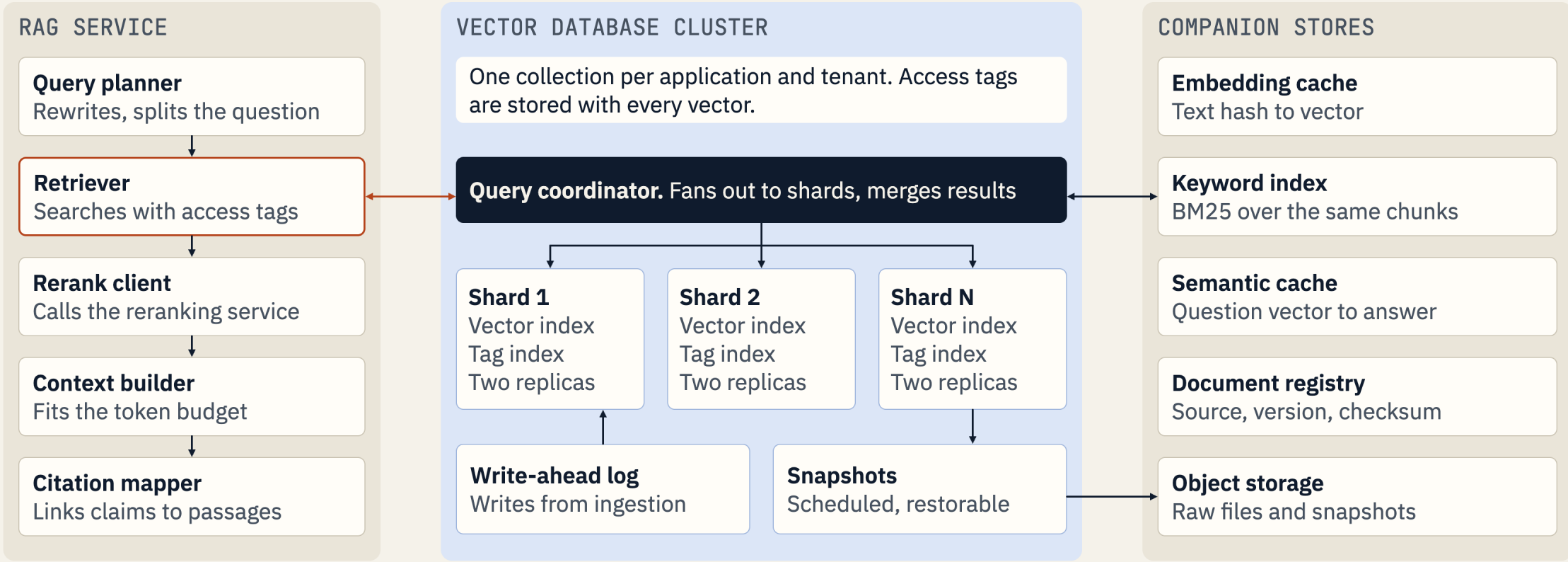
DEEP DIVE

Vector search service



ARCHITECTURE

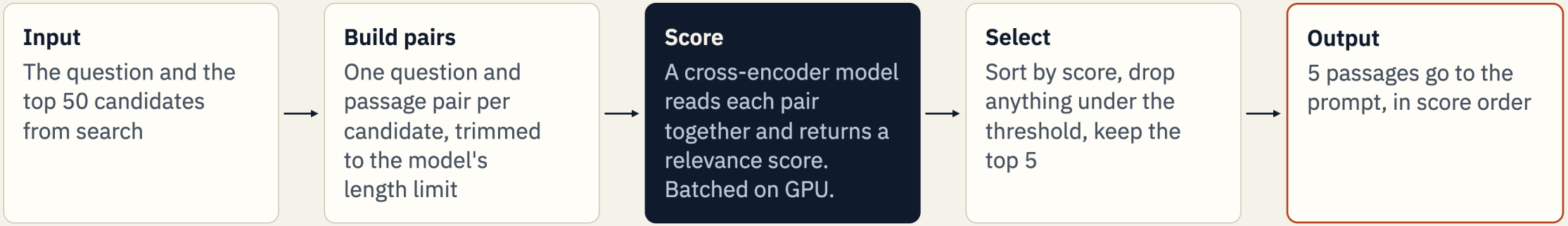
Retrieval platform architecture



Some engines hold the vector and keyword indexes together. The design is the same either way.

DEEP DIVE

Reranking service



Why a second model

Search encodes question and passage separately: fast, approximate. The reranker reads them together: slower and more precise, so it only sees the shortlist.

Fallback

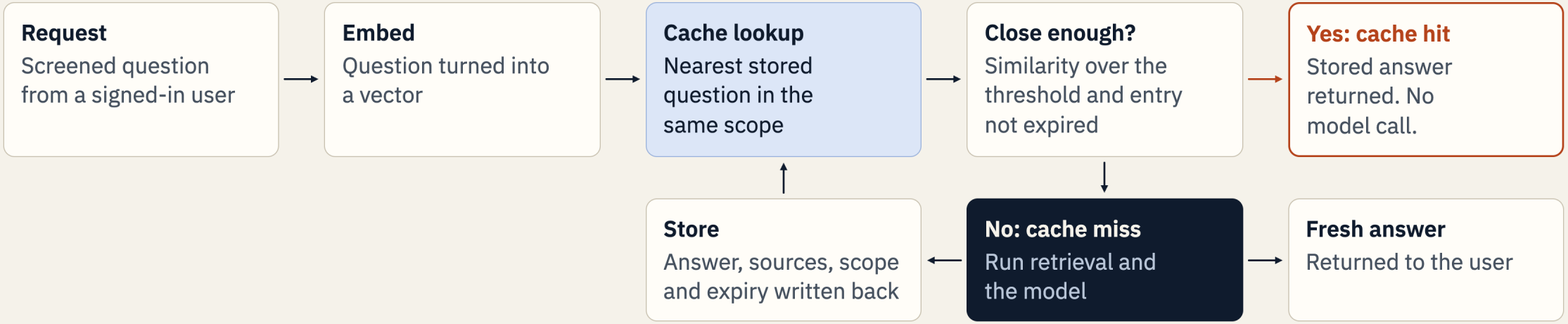
If the reranker is slow or unavailable, the search order is used and the event is logged.

Tuning

50 in and 5 out are starting defaults, adjusted for each client against the evaluation set.

DEEP DIVE

Semantic cache



Scope

An entry is reused only for the same application, access groups, prompt version and model.

Expiry

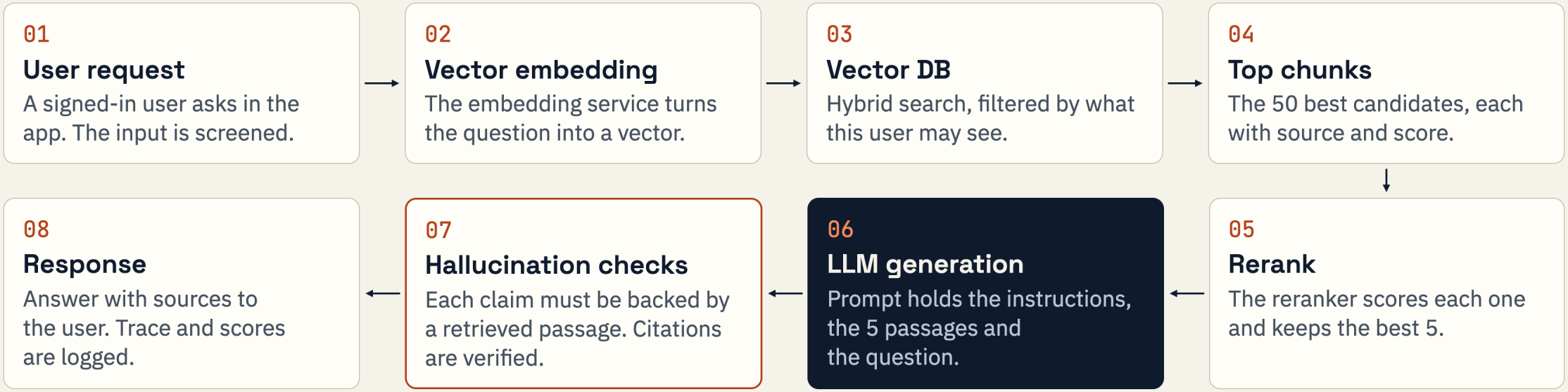
A time limit per content type. Entries are dropped when a source document changes.

Never cached

Answers with personal data, write actions, and answers that failed a check.

DEEP DIVE

Runtime execution flow



Weak retrieval: say the answer was not found. Do not guess.

Check fails: regenerate once, then return the sources only.

Model slow: fall back to a smaller model.

DEEP DIVE

Internal API gateway

CALLERS

- Application backend
- Agents and MCP tools
- Batch and pipeline jobs

GATEWAY: EVERY CALL PASSES THESE STEPS

1 Authenticate. Service identity and user token

2 Limit. Rate limits and token budgets

3 Screen. Injection and personal-data checks

4 Reuse. Exact and semantic cache lookup

5 Route. Pick the model by task, cost and policy

6 Call. Timeouts, retries and a fallback model

7 Validate. Schema and safety checks on the output

8 Record. Tokens, latency, cost and the full trace

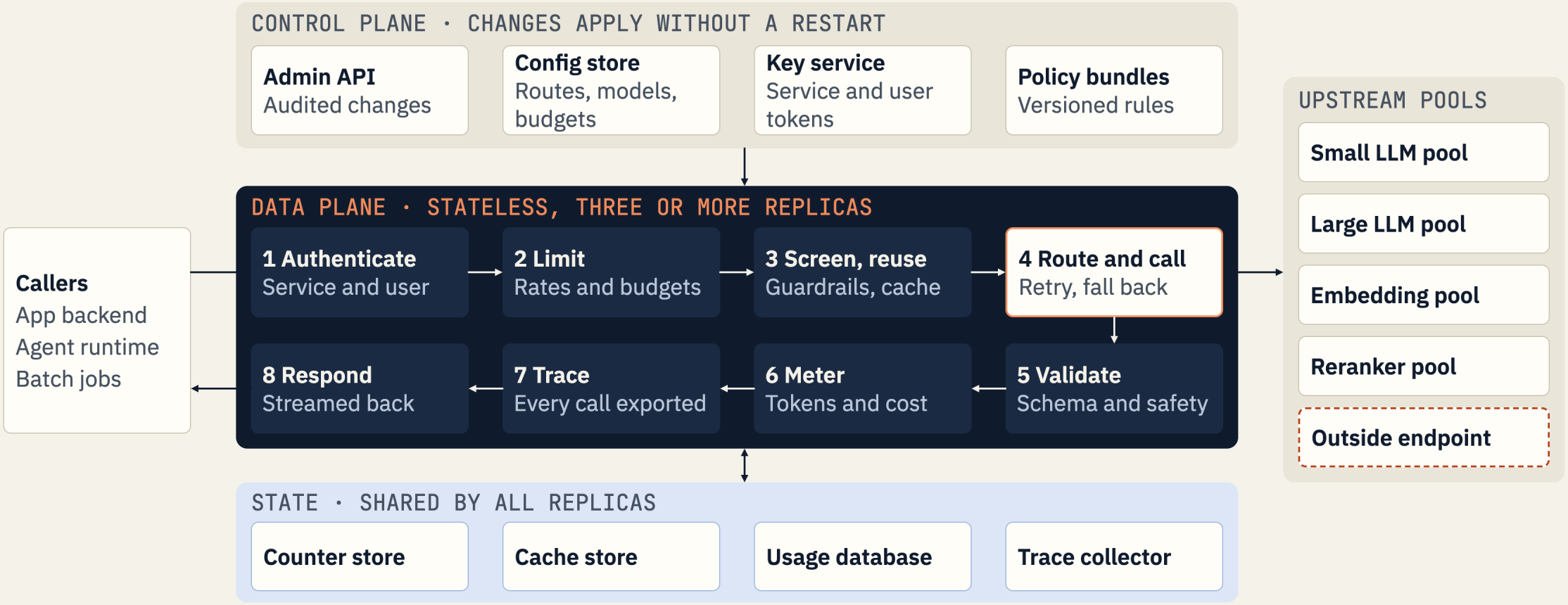
Model list, routing rules, budgets and keys are configuration here. The application does not change.

BACKENDS

- LLM serving: small, mid and large models
- Embedding service
- Reranking service
- Vector search service
- Outside endpoint, only where policy allows

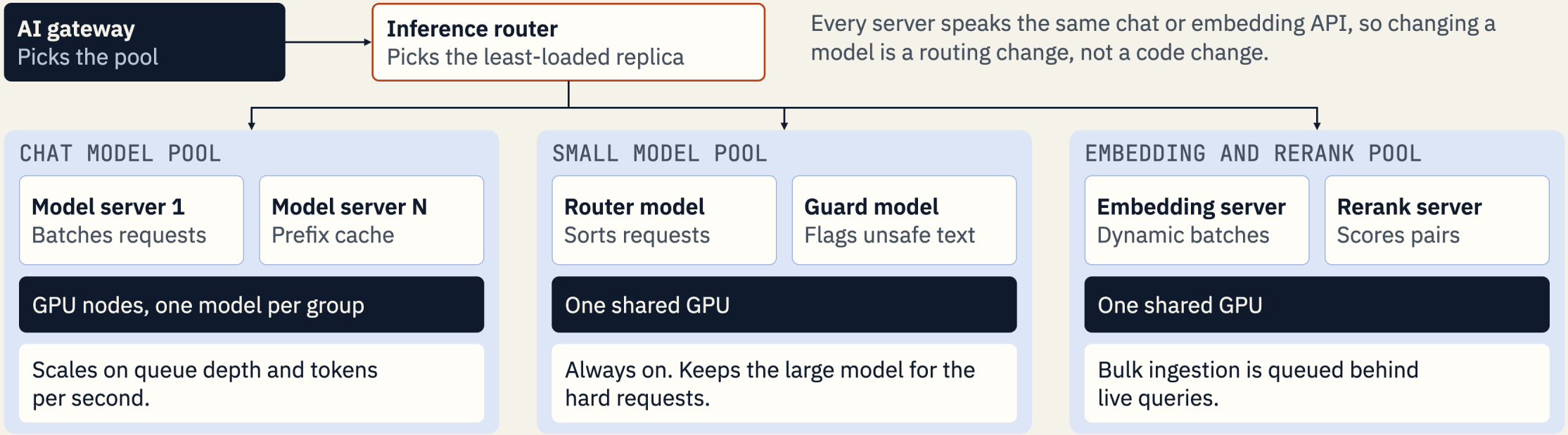
ARCHITECTURE

Gateway architecture



ARCHITECTURE

Model serving architecture



SUPPORTING SERVICES

Model store
Verified weights, cached on each node

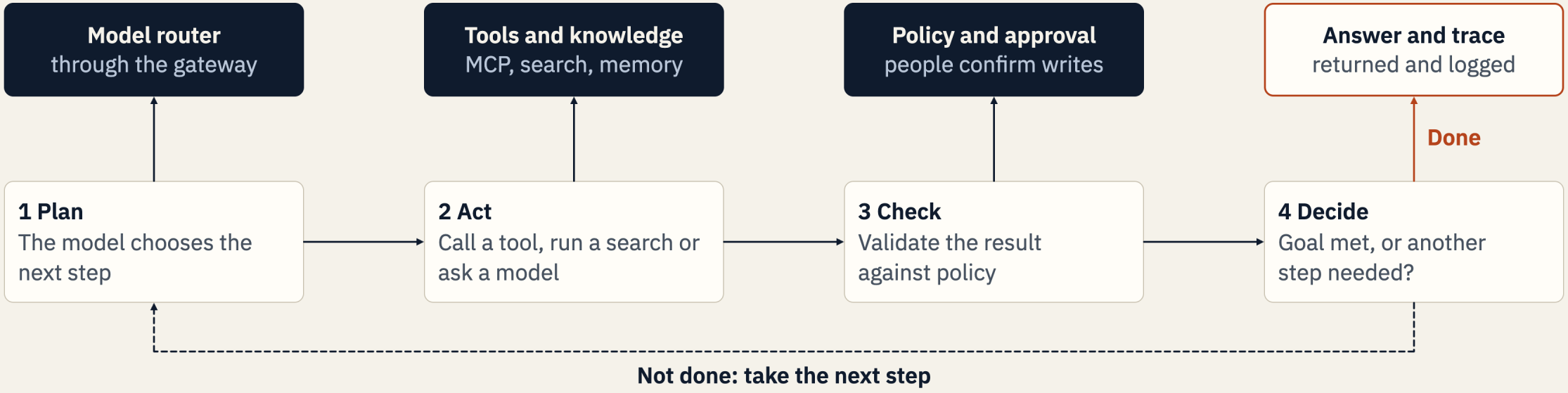
Autoscaler
Adds replicas on queue depth, keeps one warm

Rollout controller
New version runs beside the old, then traffic moves

Serving metrics
Tokens per second, queue time, GPU memory

DEEP DIVE

LLM orchestration



AROUND EVERY RUN

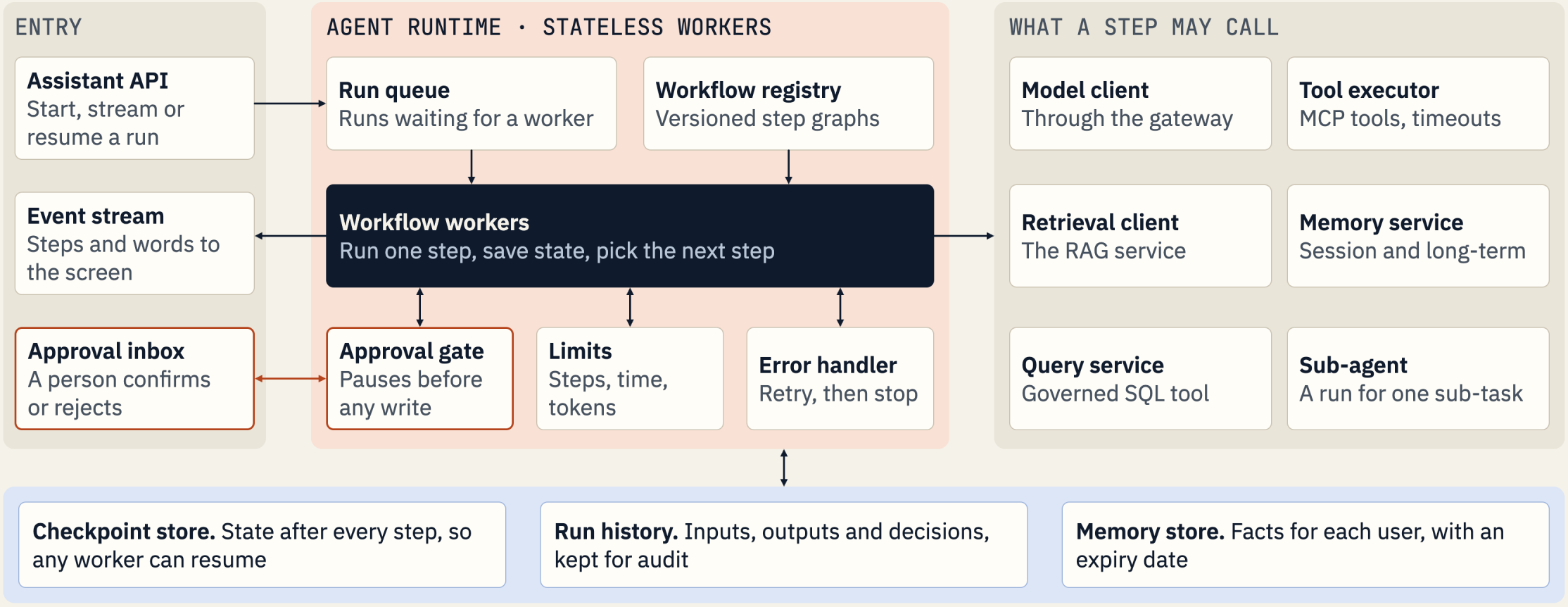
State store. A checkpoint after every step, so a run can resume.

Limits. Caps on steps, time and tokens for each run.

Patterns. Single call, retrieval chain, router, agent loop.

ARCHITECTURE

Agent runtime architecture



DEEP DIVE

LLM categories: private and public

Private. Runs inside the client's boundary. Prompts and data stay there.

Original open-weight

Published weights, run as released on the client's GPUs.

Base model: itself.

Tuned

An adapter or fine-tune trained on the client's own data.

Base model: an open-weight original.

Private endpoint

A vendor's model served inside the client's own cloud tenancy.

Base model: the vendor's.

Public. Hosted by a vendor. Prompts leave the client's boundary.

Proprietary API

The vendor's own model. Weights are not published.

Tuning only where the vendor offers it.

Hosted open-weight

An open model run by a hosting provider.

Base model: an open-weight original.

The default route is private. A public route needs a policy entry naming the classes of data allowed to leave.

DEEP DIVE

Model registry

Model slot	Availability	Type	Originality	Parameters	Base model
General model	Private, client GPUs	Chat and tool calling	Original	Mid tier	Open-weight family
Domain model	Private, client GPUs	Chat and extraction	Tuned adapter	Base plus a small adapter	The general model
Router model	Private, shared GPU	Classification	Original, quantised	Small tier	Small open-weight
Embedding and reranker	Private, shared GPU	Embedding and scoring	Original	Small tier	Open embedding family
Escalation model	Private endpoint or public API	Reasoning, long tasks	Original	Large tier, or not published	Open-weight or vendor

Working size tiers: small is under 10B parameters, mid is 10B to 40B, large is above 40B. Named models are chosen at assessment and recorded with licence and context length.

DEEP DIVE

Security and governance

CLIENT ENVIRONMENT: DATA CENTRE OR PRIVATE CLOUD

Application zone
Existing app and login
Control: user identity travels with every call

→

AI platform zone
Gateway, orchestration, search
Control: access rules, masking and guardrails enforced

→

Model zone
GPU nodes and model servers
Control: no outbound network, internal model registry

Data zone. Vector index, cache, memory and logs. Encrypted at rest and in transit, with keys held by the client.

⇢

Outside model API
Closed by default. An egress policy opens it per data class.

GOVERNANCE ACROSS ALL ZONES

Model registry. Each model approved, licensed, versioned

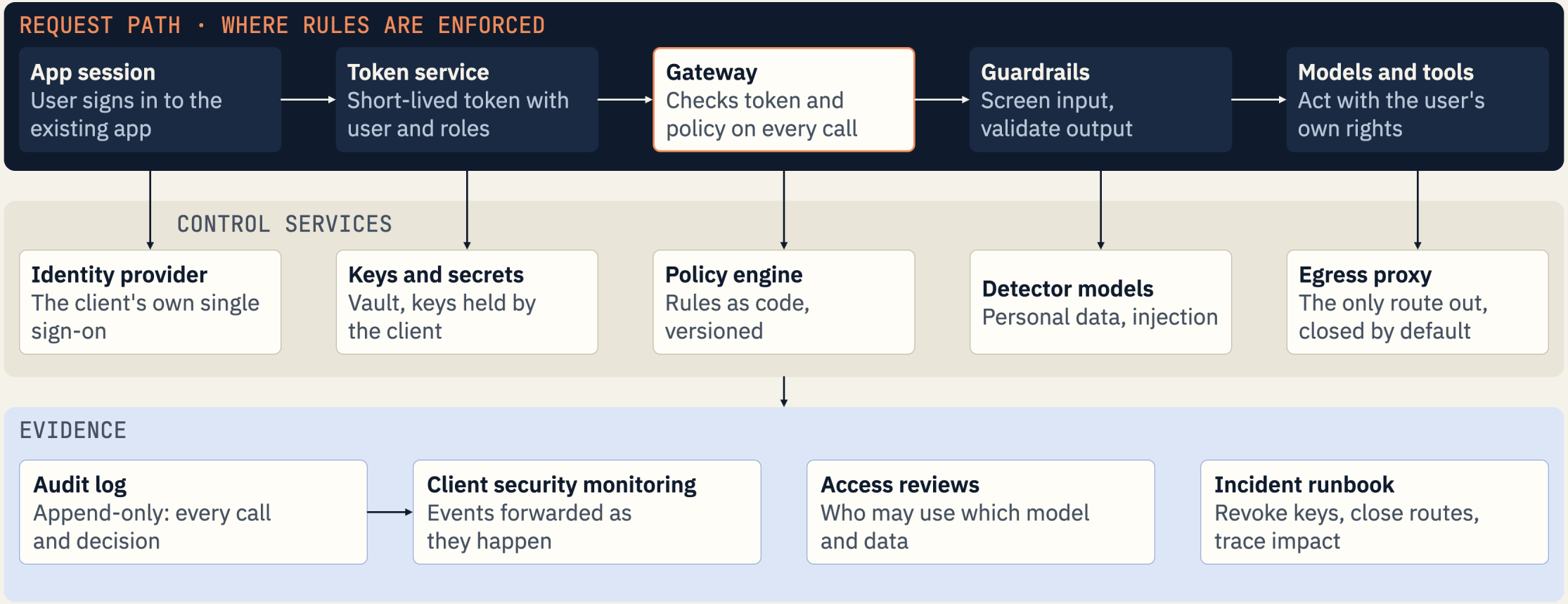
Policy as code. Who may use which model on which data

Release gate. The evaluation suite must pass

Audit. Tamper-evident log with retention rules

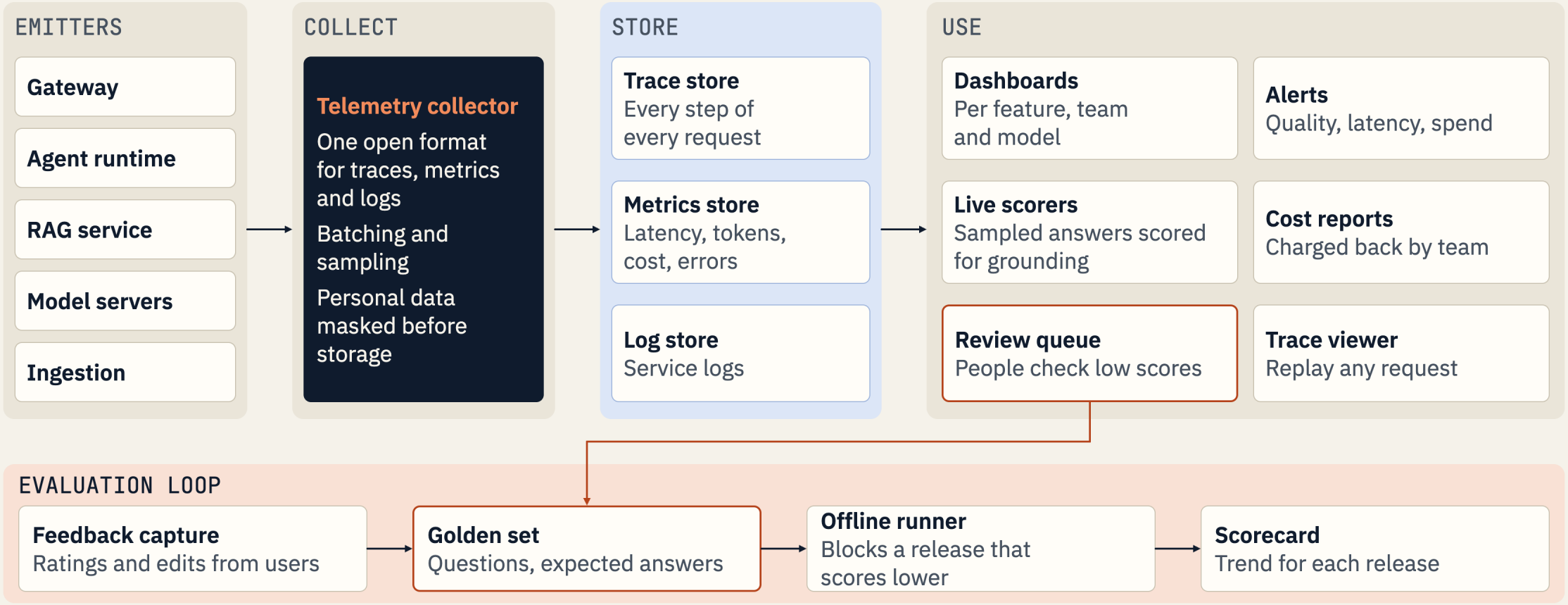
ARCHITECTURE

Security architecture



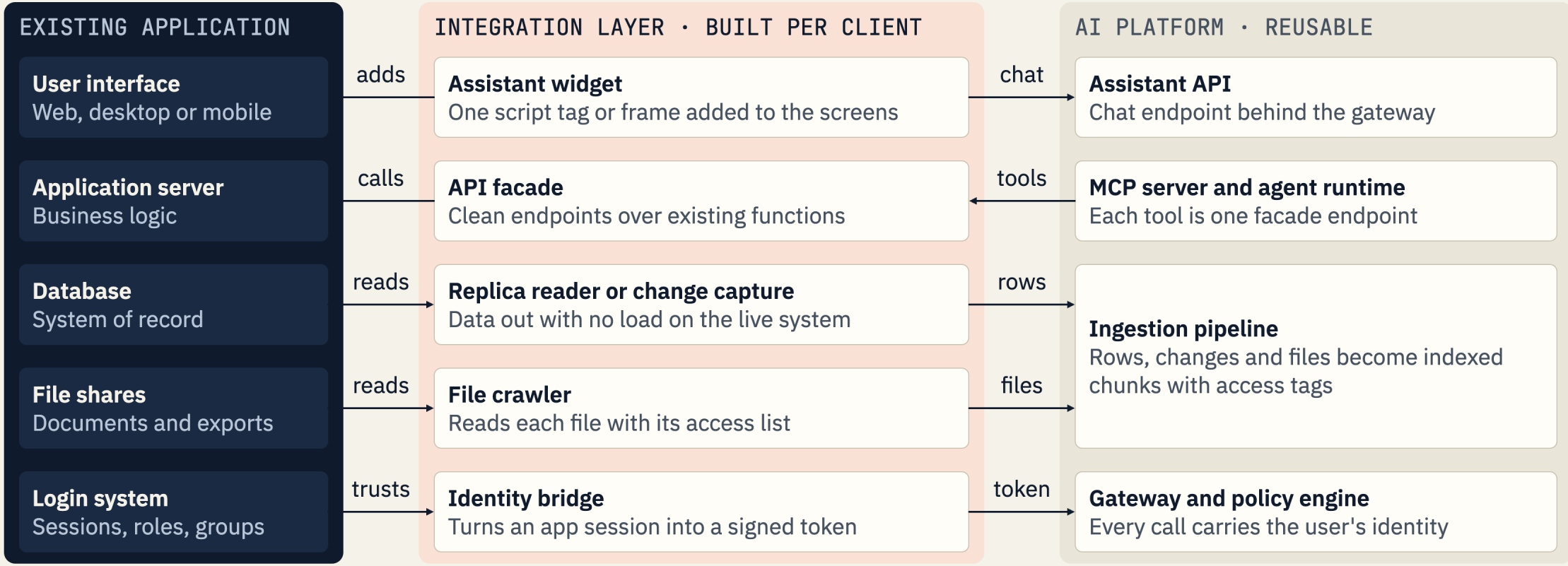
ARCHITECTURE

Observability and evaluation architecture



CLOSING THE GAPS · 1

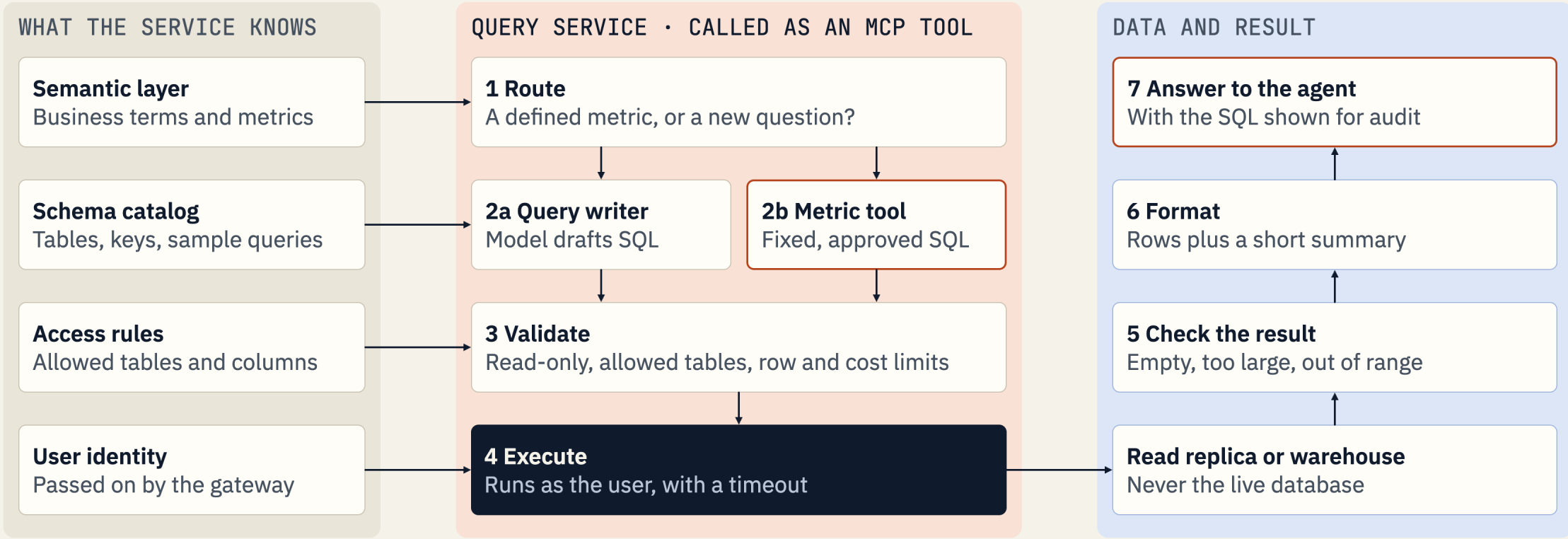
Legacy integration architecture



No usable APIs: the facade is built first. No single sign-on: the identity bridge is built first. Both come before any AI feature.

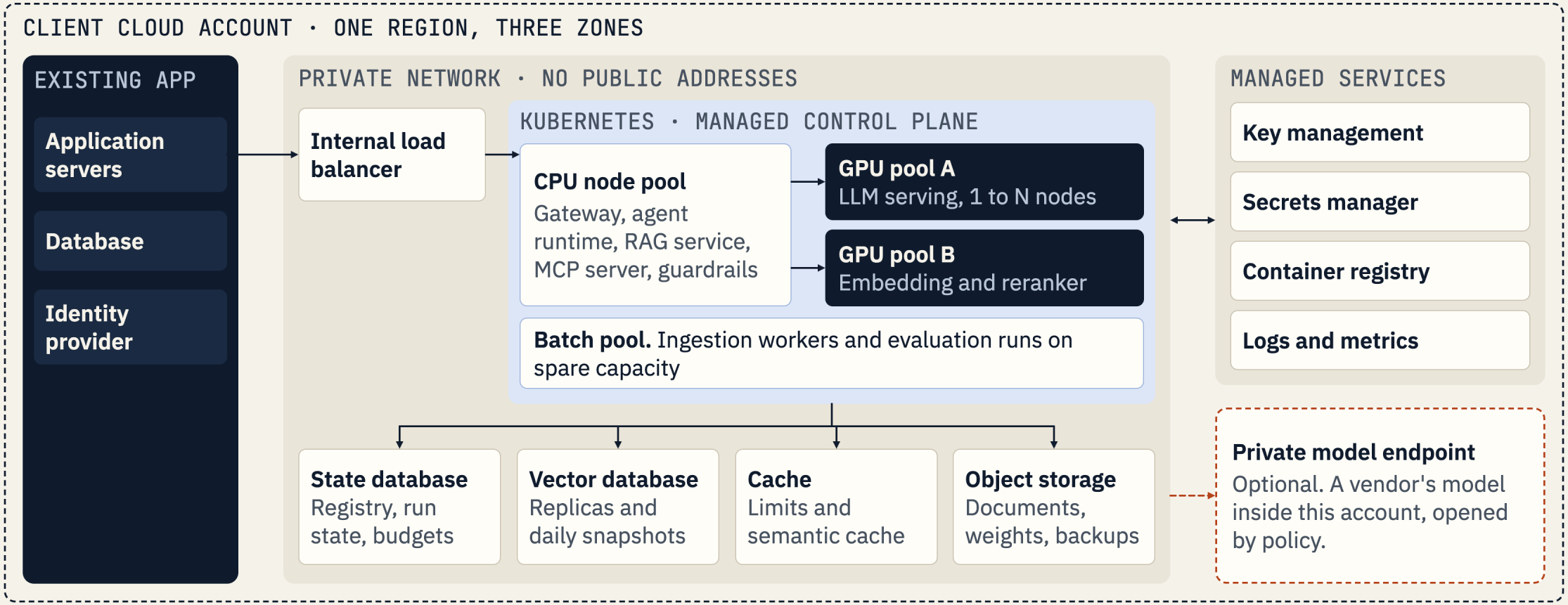
CLOSING THE GAPS · 2

Structured data access

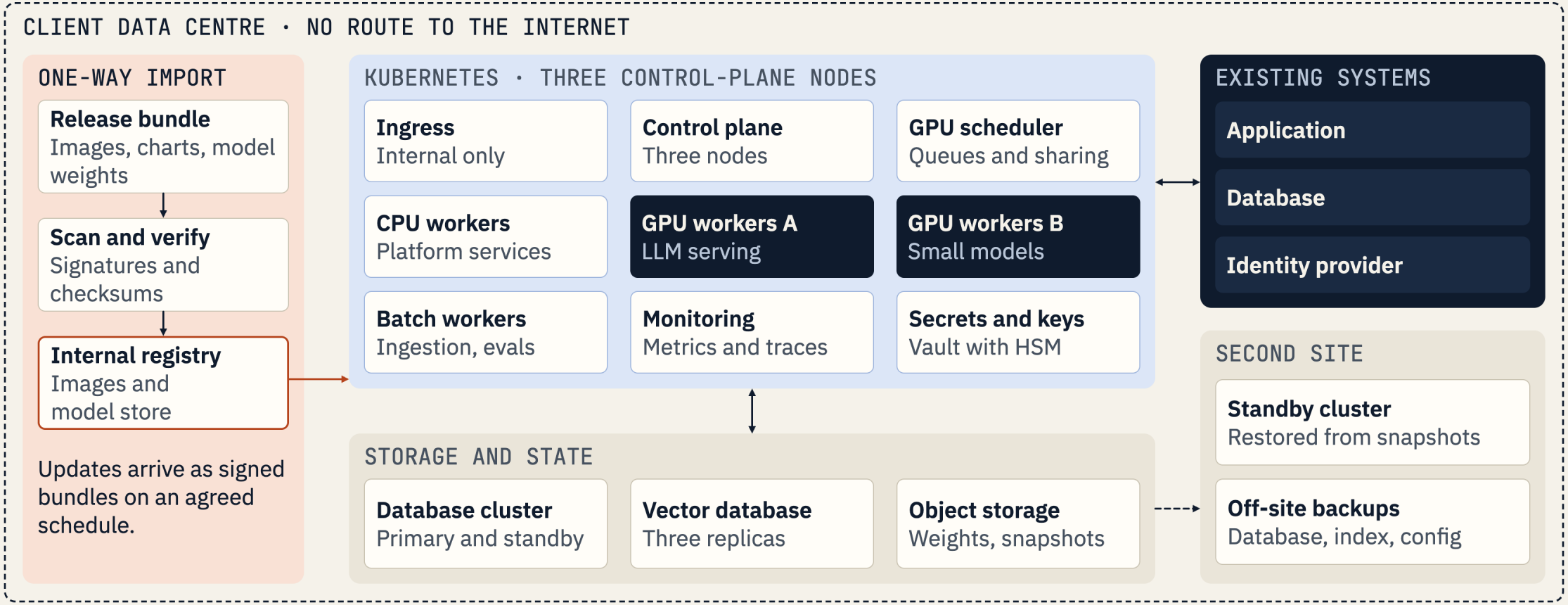


An approved metric is always preferred to generated SQL. Generated SQL is read-only, limited, checked and logged.

Deployment topology: cloud



Deployment topology: on-prem



CLOSING THE GAPS · 3

Capacity and sizing method

01 **Measure demand**
Peak users at once × requests per user per minute × tokens per request, in and out.

02 **Measure supply**
Tokens per second one GPU node sustains for the chosen model at the target latency. Taken from a load test on the client's hardware, never from a brochure.

03 **Divide, then add**
Nodes for load = demand ÷ supply. Add one node so a failure is not an outage, then headroom for growth.

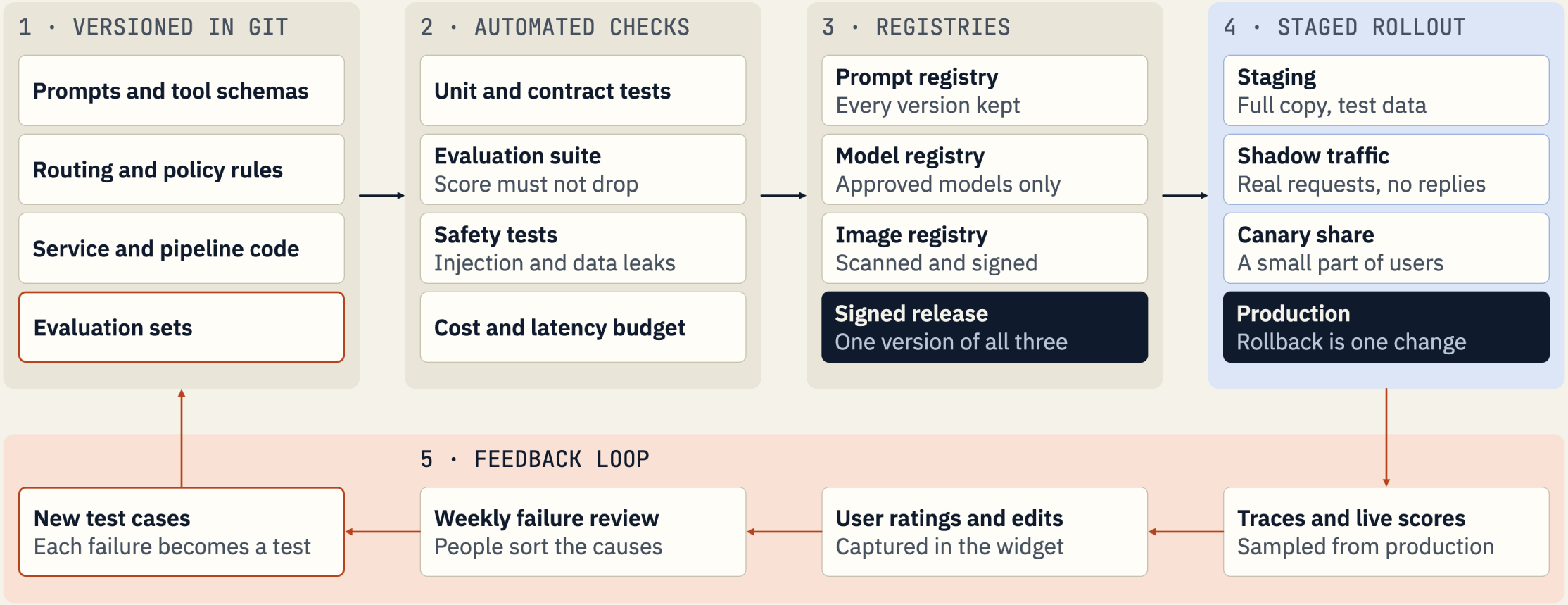
04 **Repeat for each pool**
Chat models, small models, embedding and rerank. Ingestion is sized by backlog and the nightly window.

Worksheet for one pool	Value
Peak users at once	[__]
Requests per user per minute	[__]
Tokens per request	[__]
Demand, tokens per second	product ÷ 60
Measured supply per node	[__]
Nodes for load	demand ÷ supply
Nodes to order	load + 1 + headroom

Bracketed values are filled in during the assessment. No node count is quoted before the load test.

CLOSING THE GAPS · 4

Release pipeline and feedback loop



Latency budget and response modes

Stage	Design target
Screen the input	50 ms
Embed the query	30 ms
Hybrid search	70 ms
Rerank	150 ms
Model starts answering	500 ms
Time to the first word	800 ms
Hallucination check, at the end	500 ms

Mode A: check, then send

The whole answer is written and checked before the user sees it. Safest. The user waits for the full answer.

Use for: anything that advises, decides or acts.

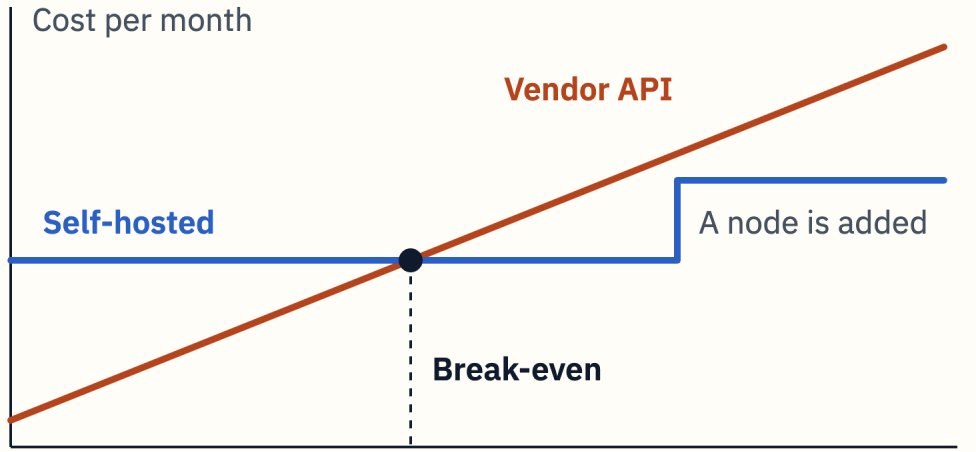
Mode B: stream, then verify

Words appear as they are written. The check runs at the end and can flag or withdraw the answer. Fastest. A wrong sentence may be seen briefly.

Use for: internal search and drafting.

Targets are design goals to confirm in load testing on the client's hardware. They are not measurements.

Cost model

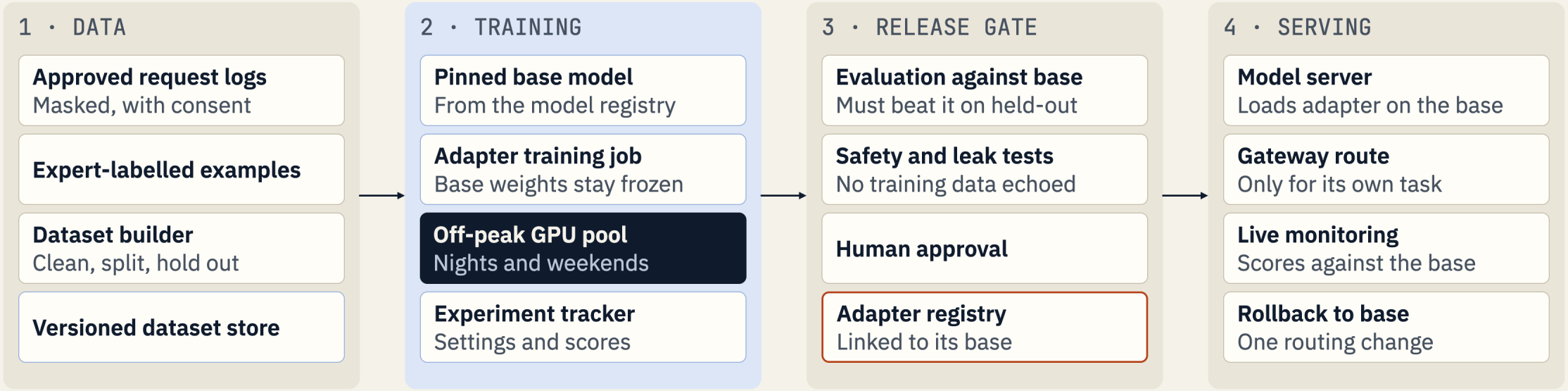


Worksheet	Value
GPU and platform cost per month	[__]
Run and support cost per month	[__]
API price for a typical request	[__]
Break-even requests per month	fixed cost ÷ API price
Client's forecast requests per month	[__]

Self-hosted is a fixed cost: hardware, platform and the people who run it.
An API is a variable cost: a price for every request.

The chart shows the shape only. Cost is one input: data control, latency and model quality can settle the choice first.

Fine-tuning pipeline



TUNE ONLY WHEN ALL THREE HOLD

A measured task has stopped improving after retrieval and prompt work.

There are enough approved examples to train on and to test with.

Someone owns retraining every time the base model changes.

Team and operating model

Role	During the build	Once it is running
Solution architect	Owns the design and the client's security review	Quarterly review of cost, quality and roadmap
Platform engineer	Cluster, GPU serving, deployment packaging	On-call, upgrades, capacity changes
Integration engineer	API facade, MCP server, identity bridge	New tools and connectors
AI engineer	Retrieval, prompts, orchestration, evaluation set	Evaluation upkeep, model and prompt changes
Data engineer	Ingestion pipelines and access-tag mapping	Pipeline health, new sources
From the client	Product owner, subject experts, security lead	Owner for feedback review and access reviews

One person can hold two roles on a first engagement. Nobody can skip the platform role: GPUs on Kubernetes need a specialist.

Headcount, on-call hours and response times are set per contract: [to be agreed].

DELIVERY

Scaling in three stages

Stage 1

Pilot

One GPU node or a private endpoint. Containers on a single host. One use case.

Move on when

Evaluation scores hold and people use it every week.

Stage 2

Production

Kubernetes with autoscaling. A redundant gateway. A second model tier for harder tasks.

Move on when

A second team or application asks for access.

Stage 3

Platform

A GPU pool shared across teams. Idle capacity scales to zero. Adapters tuned on the client's own domain.

From here

Capacity follows measured demand.

The architecture is the same at every stage. Only the footprint changes.

DELIVERY

Engagement model (proposed)

[2 weeks]

Assess

Application review, data map, shortlist of use cases, hardware sizing.

Output: a costed plan

[6 to 8 weeks]

Foundation

Gateway, search, model serving and controls, with one use case live.

Output: one feature in production

[per use case]

Expand

Further use cases and agent tools built on the same foundation.

Output: each new feature, faster

[monthly]

Run

Monitoring, model updates, capacity changes, upkeep of the test suite.

Output: recurring revenue

Durations in brackets are starting proposals for discussion. Pricing is not yet set.

FOR REVIEW

Open questions for review

01 **Which industry do we lead with?**
A general offer competes with everyone. A focused one can be found.

02 **Who is the first reference client?**
One named result will sell more than this deck.

03 **Which use case opens the conversation?**
Buyers fund an outcome. The foundation comes with it.

04 **Fixed price, or time and materials?**
Fixed price for Assess and Foundation is easier to buy and riskier to deliver.

05 **Do we run the platform after go-live?**
Recurring income, in exchange for an on-call commitment.

06 **Who supplies GPUs when the client has none?**
The client buys, we resell, or we partner with a host.